29/01/2026

TECMILENIO CAMPUS QUERETARO

DIEGO SANABRIA MARTINEZ

AVANCE DEL RETO FINAL – PROGRAMACIÓN ORIENTADA A OBJETOS

Análisis de Requerimientos

Instrucciones

Para lograr tu objetivo de desarrollar un sistema de gestión de biblioteca, debes tomar en cuenta algunos aspectos.

1. Los requisitos funcionales del sistema son los siguientes:
   a. Debe permitir registrar y consultar libros, usuarios y préstamos.
   b. Debe validar que los datos ingresados sean correctos y coherentes.
   c. Debe controlar que no se presten más libros de los que hay disponibles ni que se exceda el límite de dos ejemplares por usuario.
   d. Debe generar reportes (impresiones en pantalla) sobre el estado de la biblioteca, así como del número de libros, usuarios y préstamos.

2. Estos son los requisitos no funcionales del sistema:
   a. Debe ser fácil de usar.
   b. Debe ser robusto y manejar adecuadamente los errores.
   c. Debe ser eficiente.
   d. Debe ser escalable y permitir agregar nuevas funcionalidades en el futuro.

Requerimientos/Requisitos Funcionales

- Tener en cuenta las altas, bajas o modificaciones por usuario y préstamos.
- Asegurarse de que los datos sean coherentes.
- Mantener una lógica realista en cuanto a la cantidad de libros que pueden ser prestados y el stock dentro del programa.
- Reportes de inventario con listados y estado actual en cuanto al sistema.

Requerimientos/Requisitos no Funcionales

- Usando colecciones como ArrayList el sistema abre paso a más posibilidades como diferentes tipos de usuarios.
- Facilita el uso con cualquier interfaz de consola donde se mire claramente.

Diseño de Clases y Objetos

Clase *Libro*

- **Atributos** *isbn* (String), *título* (String), *autor* (String), *unidadesDisponibles* (int).
- **Métodos** *actualizarStock* (int cantidad), *getters/setters*.

Clase *Usuario*

- **Atributos** *idUsuario* (String), *nombre* (String) y *librosPrestados* (int).
- **Métodos** *puedePedir*() (boolean) *getters/setters*.

Clase *Prestamo*

- **Atributos** *idPrestamo* (int), *fechaInicio* (Date), *libro* (Libro), *usuario* (Usuario).
- **Métodos** *finalizarPrestamo*().

Clase *Biblioteca* (Controlador)

- **Atributos** *listaLibros*, *listaUsuarios*, *listaPrestamos*.
- **Métodos** *registrarLibro*(), *realizarPrestamo*(), *generarReporte*.

Relación Entre Clases

- La clase *Prestamo* relaciona a un *Usuario* con un *Libro*. (Asociación)
- La clase *Biblioteca* contiene colecciones o listas de *Libro*, *Usuario* y *Prestamo*. (Agregación)
- El proyecto depende de la lógica dentro de la biblioteca. (Composición)

Diagramas UML

Documentación del Diseño y Modelado

Con el objetivo de cumplir con los requisitos no funcionales se decidió este flujo para el método *realizarPrestamo* de la clase *Biblioteca*.

1. Verificar Usuario: Si existe y si tiene más de dos libros prestados a su nombre de usuario.
2. Verificar Libro: Igualmente si existe y si hay copias disponibles en el inventario.
3. Ejecución: Crear *Prestamo*

- Bajar *unidadesDisponibles* de *Libro*.
- Subir *librosPrestados* de *Usuario*.

Código Completo Comentado

Biblioteca

```java
import java.util.ArrayList;

// Crear lista de libros, usuarios y préstamos
public class Biblioteca {
    private final ArrayList<Libro> listaLibros;
    private final ArrayList<Usuario> listaUsuario;
    private final ArrayList<Prestamo> listaPrestamos;
    private int contadorPrestamos = 1;

// Constructor para inicializar las listas de la biblioteca
    public Biblioteca() {
        listaLibros = new ArrayList<>();
        listaUsuario = new ArrayList<>();
        listaPrestamos = new ArrayList<>();
    }
// Métodos para registrar libros, usuarios y realizar préstamos
    public void registrarLibro(Libro book) { listaLibros.add(book); } // Método para registrar un libro en la biblioteca
    public void registrarUsuario(Usuario user) { listaUsuario.add(user); } // Método para registrar un usuario en la biblioteca
    public void realizarPrestamo(Usuario user, Libro book) { // Método para realizar un préstamo de un libro a un usuario
        if (!user.puedePedir()) { // Verificar si el usuario puede pedir más libros
            System.out.println("El usuario " + user.getNombre() + " ha alcanzado su límite.");
            return;
        }
// Verificar si el libro tiene unidades disponibles
        if (book.getUnidadesDisponibles() <= 0) { // Si no hay unidades disponibles, mostrar un mensaje y salir del método
            System.out.println("No hay copias disponibles de " + book.getTitulo());
            return;
        }
// Crear un nuevo préstamo, actualizar el stock del libro y registrar el préstamo en el usuario
        Prestamo nuevoPrestamo = new Prestamo(contadorPrestamos++, book, user); // Crear un nuevo préstamo con un ID único, el libro y el usuario
        listaPrestamos.add(nuevoPrestamo); // Agregar el nuevo préstamo a la lista de préstamos activos
        book.actualizarStock(-1); // Actualizar el stock del libro, restando una unidad
        user.registrarPrestamo(); // Registrar el préstamo en el usuario, incrementando su contador de libros prestados
        System.out.println("Préstamo realizado. \nLibro: " + book.getTitulo() + "\nUsuario: " + user.getNombre());
    }
```

```java
public class Biblioteca {
    public void realizarPrestamo(Usuario user, Libro book) { // Método para realizar un préstamo de un libro a un usuario
        }
// Verificar si el libro tiene unidades disponibles
        if (book.getUnidadesDisponibles() <= 0) { // Si no hay unidades disponibles, mostrar un mensaje y salir del método
            System.out.println("No hay copias disponibles de " + book.getTitulo());
            return;
        }
// Crear un nuevo préstamo, actualizar el stock del libro y registrar el préstamo en el usuario
        Prestamo nuevoPrestamo = new Prestamo(contadorPrestamos++, book, user); // Crear un nuevo préstamo con un ID único, el libro y el usuario
        listaPrestamos.add(nuevoPrestamo); // Agregar el nuevo préstamo a la lista de préstamos activos
        book.actualizarStock(-1); // Actualizar el stock del libro, restando una unidad
        user.registrarPrestamo(); // Registrar el préstamo en el usuario, incrementando su contador de libros prestados
        System.out.println("Préstamo realizado. \nLibro: " + book.getTitulo() + "\nUsuario: " + user.getNombre());
    }

// Método para generar un reporte de la biblioteca
    public void generarReporte() {
        System.out.println("\n--- BIBLIOTECA ---");
        System.out.println("Total libros registrados: " + listaLibros.size());
        for ( Libro l : listaLibros) { // Mostrar cada libro con su título y unidades disponibles
            System.out.println(" - " + l.getTitulo() + " (Disponibles: " + l.getUnidadesDisponibles() + ")"); // Mostrar el título del libro y las unidades disponibl
        }
        System.out.println("Total usuarios: " + listaUsuario.size());
        System.out.println("Préstamos activos: " + listaPrestamos.size());
        System.out.println("-----------------");
    }
}
```

Estudiante

```java
// Clase Estudiante que hereda de
Usuario
class Estudiante extends Usuario {
    public Estudiante(String idUsuario,
String nombre) {
        super(idUsuario, nombre, 0); //
Llama al constructor de la clase padre
Usuario con el id, nombre y 0 libros
prestados
    }

// Método que determina si el
estudiante puede pedir más libros
(máximo 2)
    @Override
    public boolean puedePedir() {
        return  librosPrestados < 2; //
El estudiante puede pedir un libro si
tiene menos de 2 libros prestados
    }
}
```

Profesor

```java
// Clase Profesor que hereda de Usuario
class Profesor extends Usuario {
    public Profesor(String idUsuario,
String nombre) {
        super(idUsuario, nombre, 0); //
Llama al constructor de la clase padre
Usuario con el id, nombre y 0 libros
prestados
    }

// Método que determina si el profesor
puede pedir más libros prestados
    @Override
    public boolean puedePedir() {
        return librosPrestados < 5;
    }
}
```

Libro

```java
// Clase que representa un libro en la biblioteca
public class Libro {
    private String isbn;
    private String titulo;
    private String autor;
    private int unidadesDisponibles;

// Constructor para inicializar un libro con su información y stock disponible
    public Libro(String isbn, String titulo, String autor, int unidadesDisponibles) {
        this.isbn = isbn;
        this.titulo = titulo;
        this.autor = autor;
        this.unidadesDisponibles = unidadesDisponibles;
    }

// Método para actualizar el stock de un libro, sumando o restando la cantidad indicada
    public void actualizarStock(int cantidad) {
        this.unidadesDisponibles += cantidad; // Actualiza el stock del libro sumando la cantidad (puede ser positiva o negativa)
    }

// Métodos para obtener la cantidad de unidades disponibles y el título del libro
    public int getUnidadesDisponibles() { return unidadesDisponibles; } // Obtener la cantidad de unidades disponibles del libro
    public String getTitulo() { return titulo; } // Obtener el título del libro

// Método para representar el libro como una cadena de texto, mostrando su título y unidades disponibles
    @Override
    public String toString() {
        return titulo + " (Disponibles: " + unidadesDisponibles + ")";     // Representación del libro mostrando su título y unidades disponibles
    }
}
```

Prestamo

```java
import java.util.Date; // Importamos la clase Date para manejar las fechas de los préstamos

// Clase que representa un préstamo de libro a un usuario
public class Prestamo {
    private final int idPrestamo;
    private final Date fechaInicio;
    private final Libro book;
    private final Usuario user;

// Constructor de la clase Prestamo que inicializa el ID del préstamo, el libro prestado y el usuario que lo recibe
    public Prestamo(int idPrestamo, Libro book, Usuario user) {
        this.idPrestamo = idPrestamo;
        this.fechaInicio = new Date();
        this.book = book;
        this.user = user;
    }

// Método para finalizar el préstamo, actualizando el stock del libro y notificando al usuario
    public void finalizarPrestamo() {
        book.actualizarStock(1);
        user.devolverLibro();
        System.out.println("\nPréstamo finalizado " + book.getTitulo()); // Notificación de que el préstamo ha sido finalizado, mostrando el título del libro
    }
}
```

Main

```java
// Clase principal para ejecutar el programa de la biblioteca
public class Main {
    Run main | Debug main
    public static void main(String[] args) {
        Biblioteca miBiblioteca = new Biblioteca(); // Crear una instancia de la biblioteca

// Crear algunos libros con su código, título, autor y cantidad disponible
        Libro book1 = new Libro("1234", "ABC", "Juan", 2);
        Libro book2 = new Libro("456", "DFG", "Pedro", 4);

// Crear algunos usuarios, un estudiante y un profesor
        Usuario user1 = new Estudiante("A01", "Diego");
        Usuario user2 = new Profesor("P01", "Carlos");

// Registrar los libros y usuarios en la biblioteca
        miBiblioteca.registrarLibro(book1);
        miBiblioteca.registrarLibro(book2);
        miBiblioteca.registrarUsuario(user1);
        miBiblioteca.registrarUsuario(user2);

// Realizar algunos préstamos de libros a los usuarios
        miBiblioteca.realizarPrestamo(user1, book1);
        miBiblioteca.realizarPrestamo(user1, book2);
        miBiblioteca.realizarPrestamo(user2, book1);
        miBiblioteca.realizarPrestamo(user2, book2);
        miBiblioteca.generarReporte();
    }
}
```

Resultado

```
Codigo Clean + class — -zsh — 80×24

diego@192 Codigo Clean + class %
diego@192 Codigo Clean + class %
diego@192 Codigo Clean + class % java Main
Préstamo realizado.
Libro: ABC
Usuario: Diego
Préstamo realizado.
Libro: DFG
Usuario: Diego
Préstamo realizado.
Libro: ABC
Usuario: Carlos
Préstamo realizado.
Libro: DFG
Usuario: Carlos

--- BIBLIOTECA ---
Total libros registrados: 2
 - ABC (Disponibles: 0)
 - DFG (Disponibles: 2)
Total usuarios: 2
Préstamos activos: 4
------------------
diego@192 Codigo Clean + class %
```